

SVD Image Compression, Built From Scratch

A Line-by-Line and Equation-by-Equation Guide

1 Overview

This document explains `svd_image_compression.ipynb`, a notebook that derives and implements the Singular Value Decomposition (SVD) *without* calling any library eigensolver or SVD routine (no `np.linalg.svd`, no `np.linalg.eig`), and uses that decomposition to compress and decompress an image. Every code block below is paired with the linear algebra it implements.

The notebook has one goal: take an image, treat it as a numerical matrix, and show that most of its information lives in a handful of dominant directions. Compression exploits this by keeping only those directions.

The pipeline is:

Step 1. Implement the **Jacobi eigenvalue algorithm** for symmetric matrices, entirely from elementary matrix operations.

Step 2. Use it to eigendecompose $A^T A$, and from that build the full SVD $A = U \Sigma V^T$.

Step 3. Load an image and run our from-scratch SVD on its pixel matrix.

Step 4. Compress: truncate to the top- k singular triplets.

Step 5. Decompress: multiply the truncated factors back together.

Step 6. Measure quality (PSNR) and storage savings (compression ratio) across a range of k .

2 Introduction to Linear Algebra

This section is a fast, self-contained refresher on the linear algebra needed to understand SVD. If you're already comfortable with vectors, matrices, and eigenvectors, skip ahead to Section 3.

2.1 Vectors and Matrices

A **vector** $x \in \mathbb{R}^n$ is an ordered list of n real numbers, thought of as a point (or arrow) in n -dimensional space. A **matrix** $A \in \mathbb{R}^{m \times n}$ is a rectangular array of numbers with m rows and n columns. In this document, a grayscale image *is* a matrix: A_{ij} is the brightness of the pixel at row i , column j .

The most important operation is **matrix-vector multiplication** Ax : it takes a vector in $\mathbb{R}^n$ and produces a vector in $\mathbb{R}^m$. Crucially, this map is *linear*: $A(\alpha x + \beta y) = \alpha Ax + \beta Ay$. So every matrix can be thought of as a linear transformation — a way of stretching, rotating, and reflecting space.

2.2 Length, Angle, and Orthogonality

The **norm** (length) of a vector is $\|x\|_2 = \sqrt{x^T x} = \sqrt{\sum_i x_i^2}$. The **inner product** $x^T y = \sum_i x_i y_i$ measures how aligned two vectors are; two vectors are **orthogonal** if $x^T y = 0$ (they point in perpendicular directions).

A set of vectors is **orthonormal** if they are mutually orthogonal and each has unit length. A square matrix Q whose columns are orthonormal is called an **orthogonal matrix**, and satisfies

$$Q^T Q = Q Q^T = I. \quad (1)$$

Geometrically, multiplying by an orthogonal matrix is a rotation and/or reflection: it changes direction but never length, since $\|Qx\|_2^2 = x^T Q^T Q x = x^T x = \|x\|_2^2$. Orthogonal matrices appear throughout SVD as the U and V factors.

2.3 Eigenvalues and Eigenvectors

For a square matrix $S \in \mathbb{R}^{n \times n}$, a nonzero vector v is an **eigenvector** with **eigenvalue** λ if

$$Sv = \lambda v. \quad (2)$$

In words: v is a special direction that S only stretches (by factor λ) rather than rotates. Most $n \times n$ matrices have n such eigenvector directions (counted with multiplicity), and collecting them into a matrix $V = [v_1 \cdots v_n]$ with eigenvalues on a diagonal matrix Λ gives the **eigendecomposition** $SV = V\Lambda$, or $S = V\Lambda V^{-1}$ when V is invertible.

Theorem 1 (Spectral theorem). *If S is real and **symmetric** ($S = S^T$), its eigenvectors can always be chosen real and orthonormal, so $V^{-1} = V^T$ and*

$$S = V\Lambda V^T. \quad (3)$$

This is the single most important fact this document relies on: it lets us turn "decompose a symmetric matrix" into "find an orthogonal matrix V and a diagonal matrix Λ ," which is precisely what the Jacobi algorithm in Section 4.3 computes.

Source page: Goodfellow, Bengio, and Courville, Deep Learning (MIT Press, 2016), Section 2.7 "Eigendecomposition," pp. 40–43.

2.4 Why SVD Needs More Than Eigendecomposition

Eigendecomposition (Equation (3)) only applies to *square, symmetric* matrices. An image is generally **rectangular** ($m \neq n$), and even square images aren't symmetric. SVD is the generalization of eigendecomposition to *any* matrix, achieved by using *two* orthogonal matrices (U and V) instead of one, and by applying eigendecomposition not to A itself but to the symmetric matrix $A^T A$ built from it (Section 4.2).

3 How SVD Works

3.1 The Geometric Picture

Every matrix A , no matter its shape, can be understood as three elementary steps applied one after another:

1. **Rotate** (by V^T): reorient the input space's axes.
2. **Scale** (by Σ): stretch or shrink along each axis by σ_i , possibly changing dimension.
3. **Rotate** (by U): reorient into the output space.

Concretely, $Ax = U\Sigma V^T x = U(\Sigma(V^T x))$: first V^T rotates x , then Σ scales each coordinate independently, then U rotates the result. A unit circle (or sphere) fed through A always comes out as an ellipse (or ellipsoid) — the σ_i are exactly the lengths of that ellipse's axes, and the v_i/u_i are the directions of those axes before/after the transformation. This is the sense in which σ_i measures "how much of A " lives along direction v_i : large σ_i means A stretches that direction a lot (a lot of "energy"); small σ_i means that direction barely matters.

3.2 Why This Helps Compress an Image

Treat the image matrix A as data rather than a transformation. Writing $A = \sum_i \sigma_i u_i v_i^T$ (Equation (5) below) expresses the image as a weighted sum of simple rank-1 "template" images $u_i v_i^T$, ordered from most to least important by σ_i . Natural images are highly structured (smooth gradients, repeated edges, correlated rows/columns), so only a handful of these templates are needed to reconstruct something visually close to the original — which is exactly what Figure 2 demonstrates: even $k = 16$ out of 126 possible terms already produces a recognizable image.

3.3 The Three Ideas This Document Chains Together

1. **Spectral theorem** (Equation (3)): any symmetric matrix diagonalizes with an orthogonal eigenvector matrix.
2. $A^T A$ is **symmetric** for *any* A , so the spectral theorem applies to it even when A itself is rectangular or asymmetric.
3. **Eckart–Young–Mirsky**: truncating that decomposition to its largest terms gives the best possible low-rank summary, in a precise, provable sense.

Together these three ideas are the entire mathematical content of this project; everything below is turning them into working code.

4 Mathematical Background

4.1 What SVD Says

Definition 1 (Singular Value Decomposition). *For any real matrix $A \in \mathbb{R}^{m \times n}$, there exist orthogonal matrices $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{n \times n}$, and a diagonal matrix $\Sigma \in \mathbb{R}^{m \times n}$ with non-negative entries $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(m,n)} \geq 0$ such that*

$$A = U\Sigma V^T. \tag{4}$$

The columns of U are the left singular vectors, the columns of V are the right singular vectors, and the σ_i are the singular values.

Source page: Goodfellow, Bengio, and Courville, Deep Learning (MIT Press, 2016), Section 2.8 "Singular Value Decomposition," p. 44.

Equivalently, A can be written as a sum of rank-1 matrices:

$$A = \sum_{i=1}^r \sigma_i u_i v_i^T, \quad r = \text{rank}(A). \quad (5)$$

Because the σ_i are sorted in decreasing order, the early terms in Equation (5) contribute the most "energy" to A . This is exactly what makes SVD useful for compression: dropping the small- σ_i terms removes the least important information first.

Theorem 2 (Eckart–Young–Mirsky). *Among all matrices of rank $\leq k$, the truncated SVD $A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$ is the best possible rank- k approximation of A , in both the Frobenius norm and the spectral norm:*

$$A_k = \arg \min_{\text{rank}(B) \leq k} \|A - B\|_F. \quad (6)$$

This theorem is the mathematical justification for the entire compression scheme: truncating the SVD is not just a way to compress the image, it is provably the *optimal* low-rank compression with respect to squared pixel error.

Source page: the connection between truncated SVD and optimal low-rank approximation underlies the PCA discussion in Goodfellow, Bengio, and Courville, Deep Learning (MIT Press, 2016), Section 2.12 "Example: Principal Components Analysis," pp. 48–51.

4.2 From Eigendecomposition to SVD

We never compute U , Σ , V directly. Instead we build them from the eigendecomposition of a symmetric matrix, as follows.

Source page: Goodfellow, Bengio, and Courville, Deep Learning (MIT Press, 2016), Section 2.7 "Eigendecomposition," p. 42, and Section 2.8 "Singular Value Decomposition," pp. 44–45.

Consider $A^T A \in \mathbb{R}^{n \times n}$. It is symmetric and positive semi-definite, since $x^T (A^T A) x = \|Ax\|_2^2 \geq 0$ for all x . Substituting Equation (4):

$$A^T A = (U \Sigma V^T)^T (U \Sigma V^T) = V \Sigma^T U^T U \Sigma V^T = V (\Sigma^T \Sigma) V^T, \quad (7)$$

using $U^T U = I$ (orthogonality of U). Since $\Sigma^T \Sigma$ is diagonal with entries σ_i^2 , Equation (7) says exactly that

$$A^T A = V \Lambda V^T, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_n), \quad \lambda_i = \sigma_i^2. \quad (8)$$

So: **the eigenvectors of $A^T A$ are the right singular vectors of A , and the singular values are the square roots of the eigenvalues of $A^T A$:**

$$\sigma_i = \sqrt{\lambda_i}. \quad (9)$$

Once V and Σ are known, the left singular vectors follow from Equation (4) by right-multiplying by V : $AV = U\Sigma$, so for each column i with $\sigma_i > 0$,

$$u_i = \frac{1}{\sigma_i} A v_i. \quad (10)$$

(If $\sigma_i = 0$, the corresponding u_i does not affect the reconstruction and is left as zero.)

This is the entire derivation used in the code: get eigenpairs of $A^T A \rightarrow$ singular values via Equation (9) $\rightarrow$ left singular vectors via Equation (10). For efficiency, if A has more rows than columns ($m > n$) we eigendecompose $A^T A$ (size $n \times n$); if it has more columns than rows we instead eigendecompose AA^T (size $m \times m$, symmetric by the same argument) and recover V analogously, so we always diagonalize the *smaller* of the two Gram matrices.

4.3 The Jacobi Eigenvalue Algorithm

Everything above assumed we can eigendecompose a symmetric matrix. We compute that ourselves using the **Jacobi eigenvalue algorithm**, one of the oldest (1846) and most numerically stable eigenvalue methods.

4.3.1 Givens Rotations

For a symmetric matrix $S \in \mathbb{R}^{n \times n}$, pick an off-diagonal entry S_{pq} ($p \neq q$) we would like to zero out. Define the Givens rotation matrix $J(p, q, \theta)$: the identity matrix except

$$J_{pp} = J_{qq} = \cos \theta, \quad J_{pq} = \sin \theta, \quad J_{qp} = -\sin \theta. \quad (11)$$

J is orthogonal ($J^T J = I$) for any θ , since it is a rotation in the (p, q) -plane. Applying the similarity transform

$$S' = J^T S J \quad (12)$$

preserves symmetry and preserves eigenvalues (similarity transforms don't change the spectrum), but changes the off-diagonal structure. Expanding Equation (12) for entry (p, q) gives

$$S'_{pq} = (S_{pp} - S_{qq}) \sin \theta \cos \theta + S_{pq}(\cos^2 \theta - \sin^2 \theta). \quad (13)$$

Setting $S'_{pq} = 0$ and solving for θ yields the closed-form rotation angle

$$\theta = \frac{1}{2} \arctan \left(\frac{2S_{pq}}{S_{pp} - S_{qq}} \right). \quad (14)$$

(When $S_{pp} = S_{qq}$, take $\theta = \pi/4$.)

4.3.2 Cyclic Sweeps

A single rotation zeroes S_{pq} , but earlier zeroed entries can become non-zero again after later rotations. So we repeat: sweep through *all* pairs (p, q) with $p < q$, applying a rotation to each, and repeat these sweeps until the total off-diagonal energy

$$\text{off}(S) = \sqrt{2 \sum_{p < q} S_{pq}^2} \quad (15)$$

falls below a tolerance. This is the **cyclic Jacobi method**. It is guaranteed to converge (each rotation strictly decreases $\text{off}(S)$ for the pair it targets, and the sequence of matrices stays bounded), and in practice converges quadratically once off-diagonal entries are small.

We accumulate every rotation applied into an orthogonal matrix V (initialized to I):

$$V \leftarrow V J^{(1)} J^{(2)} \dots \quad (16)$$

so that after convergence $S^{(\text{final})} \approx \Lambda$ (diagonal) and

$$S = V \Lambda V^T, \quad (17)$$

which is exactly the symmetric eigendecomposition we need for Equation (8).

Algorithm 1 Cyclic Jacobi Eigenvalue Algorithm

Require: symmetric $S \in \mathbb{R}^{n \times n}$, tolerance ϵ , max sweeps T

```
1:  $A \leftarrow S, \quad V \leftarrow I_n$ 
2: for  $t = 1, \dots, T$  do
3:   if  $\text{off}(A) < \epsilon$  then break
4:   end if
5:   for  $p = 1, \dots, n-1$  do
6:     for  $q = p+1, \dots, n$  do
7:       if  $|A_{pq}|$  negligible then continue
8:       end if
9:        $\theta \leftarrow \frac{1}{2} \arctan(2A_{pq}/(A_{pp} - A_{qq}))$ 
10:       $c \leftarrow \cos \theta, \quad s \leftarrow \sin \theta$ 
11:      rows  $p, q$  of  $A \leftarrow$  rotate by  $(c, s)$   $\triangleright A \leftarrow J^T A$ 
12:      cols  $p, q$  of  $A \leftarrow$  rotate by  $(c, s)$   $\triangleright A \leftarrow AJ$ 
13:      cols  $p, q$  of  $V \leftarrow$  rotate by  $(c, s)$   $\triangleright V \leftarrow VJ$ 
14:     end for
15:   end for
16: end for
17: return  $\text{diag}(A)$  (eigenvalues),  $V$  (eigenvectors as columns)
```

Note: in the code, the row/column rotation is applied directly with vector operations (`c*row_p + s*row_q`, etc.) rather than by forming the full matrix J and multiplying it out — mathematically identical to Equation (12), but touches only $O(n)$ entries per rotation instead of $O(n^2)$.

4.4 Truncation and Compression

Given the full SVD $A = U\Sigma V^T$, define the rank- k truncation

$$A_k = U_{:,1:k} \Sigma_{1:k,1:k} V_{1:k,:}^T = \sum_{i=1}^k \sigma_i u_i v_i^T. \quad (18)$$

By the Eckart–Young–Mirsky theorem (Equation (6)), this is the best possible rank- k approximation of the image.

Storage cost. An $m \times n$ image costs mn numbers to store. The truncated factors cost

$$\underbrace{mk}_{U_k} + \underbrace{k}_{\Sigma_k} + \underbrace{nk}_{V_k^T} = k(m+n+1) \quad (19)$$

numbers. The **compression ratio** is therefore

$$\text{ratio}(k) = \frac{mn}{k(m+n+1)}. \quad (20)$$

This is > 1 (net savings) whenever $k < \frac{mn}{m+n+1}$, which for a square $n \times n$ image simplifies to roughly $k < n/2$.

Quality metric. Reconstruction quality is measured with **Peak Signal-to-Noise Ratio**:

$$\text{MSE} = \frac{1}{mn} \sum_{i,j} (A_{ij} - (A_k)_{ij})^2, \quad \text{PSNR} = 20 \log_{10}(255) - 10 \log_{10}(\text{MSE}) \text{ [dB]}. \quad (21)$$

Higher PSNR means the compressed image is closer to the original; values above roughly 30–35 dB are typically visually near-lossless for 8-bit images.

Energy captured. The fraction of "energy" (squared Frobenius norm) retained by keeping the top k singular values is

$$\text{energy}(k) = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2}, \quad (22)$$

which follows from $\|A\|_F^2 = \sum_i \sigma_i^2$ (a direct consequence of Equation (5) and orthogonality of the u_i, v_j). The notebook uses Equation (22) to pick k values that retain, e.g., 90% or 99% of the image's energy.

5 Code Walkthrough

This section maps each notebook code cell to the math above. Only the core functions are reproduced; the notebook also contains plotting/demo cells that call these functions.

5.1 Eigendecomposition: `jacobi_eigenvalue`

Implements Algorithm 1 directly. Row/column rotations correspond to Equation (12); the convergence check uses Equation (15); the rotation angle is Equation (14).

```
def jacobi_eigenvalue(S, tol=1e-10, max_sweeps=100):
    A = S.copy().astype(np.float64)
    n = A.shape[0]
    V = np.eye(n)

    for sweep in range(max_sweeps):
        off_diag_energy = np.sqrt(2.0 * np.sum(np.triu(A, 1) ** 2))
        if off_diag_energy < tol:
            break

        for p in range(n - 1):
            for q in range(p + 1, n):
                apq = A[p, q]
                if abs(apq) < 1e-14:
                    continue

                app, aqq = A[p, p], A[q, q]
                if abs(app - aqq) < 1e-14:
                    theta = np.pi / 4.0 if apq > 0 else -np.pi / 4.0
                else:
                    theta = 0.5 * np.arctan2(2.0 * apq, app - aqq)

                c, s = np.cos(theta), np.sin(theta)

                # A <- J^T A (rotate rows p, q)
```

```

row_p, row_q = A[p, :].copy(), A[q, :].copy()
A[p, :] = c * row_p + s * row_q
A[q, :] = -s * row_p + c * row_q

# A <- A J (rotate cols p, q)
col_p, col_q = A[:, p].copy(), A[:, q].copy()
A[:, p] = c * col_p + s * col_q
A[:, q] = -s * col_p + c * col_q

# V <- V J (accumulate rotation)
vcol_p, vcol_q = V[:, p].copy(), V[:, q].copy()
V[:, p] = c * vcol_p + s * vcol_q
V[:, q] = -s * vcol_p + c * vcol_q

eigvals = np.diag(A).copy()
return eigvals, V

```

Correspondence to the math:

- Lines computing `theta` $\leftrightarrow$ Equation (14).
- The three paired row/column updates $\leftrightarrow$ Equation (12), expanded into vector form rather than full matrix products for speed.
- `off_diag_energy` $\leftrightarrow$ Equation (15), the stopping criterion.
- Returned `eigvals, V` $\leftrightarrow$ Λ, V in Equation (17).

5.2 Building SVD: `svd_from_scratch`

Implements Equations (7)–(10), choosing the smaller Gram matrix ($A^T A$ or AA^T) to eigendecompose for efficiency.

```

def svd_from_scratch(A, verbose=False):
    A = A.astype(np.float64)
    m, n = A.shape
    r = min(m, n)

    if n <= m:
        AtA = A.T @ A # n x n, Eq. (7)
        eigvals, V = jacobi_eigenvalue(AtA)
        order = np.argsort(eigvals)[::-1] # sort descending: sigma_1 >= sigma_2 >= ...
        eigvals, V = eigvals[order], V[:, order]
        eigvals = np.clip(eigvals[:r], 0, None)
        sigma = np.sqrt(eigvals) # Eq. (8): sigma_i = sqrt(lambda_i)
        V = V[:, :r]

        U = np.zeros((m, r))
        for i in range(r):
            if sigma[i] > 1e-10:
                U[:, i] = (A @ V[:, i]) / sigma[i] # Eq. (9): u_i = A v_i / sigma_i
            else:
                U[:, i] = 0.0
    else:
        AAt = A @ A.T # m x m (smaller here), symmetric analog of Eq. (7)

```



```

eigvals, U = jacobi_eigenvalue(AAt)
order = np.argsort(eigvals)[::-1]
eigvals, U = eigvals[order], U[:, order]
eigvals = np.clip(eigvals[:r], 0, None)
sigma = np.sqrt(eigvals)
U = U[:, :r]

V = np.zeros((n, r))
for i in range(r):
    if sigma[i] > 1e-10:
        V[:, i] = (A.T @ U[:, i]) / sigma[i]
    else:
        V[:, i] = 0.0

return U, sigma, V.T

```

Correspondence to the math:

- $A.T @ A \leftrightarrow A^T A$ in Equation (7).
- $\text{np.sqrt}(\text{eigvals}) \leftrightarrow$ Equation (9).
- $(A @ V[:, i]) / \text{sigma}[i] \leftrightarrow$ Equation (10).
- Sorting eigenvalues descending enforces $\sigma_1 \geq \sigma_2 \geq \dots$, required by the SVD definition.
- The clip-to-zero ($\text{sigma}[i] > 1\text{e-}10$) handles the rank-deficient case where $\sigma_i \approx 0$, avoiding division by (near) zero.
- Choosing $A^T A$ ($n \times n$) vs. AA^T ($m \times m$) whichever is smaller reduces the Jacobi algorithm's $O(n^3)$ -per-sweep cost by eigendecomposing the smaller Gram matrix.

5.3 Compression / Decompression

Directly implements Equations (18), (19)–(21).

```

def compress(U, S, Vt, k):
    return U[:, :k].copy(), S[:k].copy(), Vt[:, k, :].copy()

def decompress(Uk, Sk, Vtk):
    return Uk @ np.diag(Sk) @ Vtk # Eq. (16): A_k = U_k Sigma_k V_k^T

def compression_ratio(m, n, k):
    original = m * n
    compressed = k * (m + n + 1) # Eq. (17)
    return original / compressed # Eq. (18)

def psnr(original, reconstructed, max_val=255.0):
    mse = np.mean((original - reconstructed) ** 2) # Eq. (19), left
    if mse == 0:
        return float('inf')
    return 20 * np.log10(max_val) - 10 * np.log10(mse) # Eq. (19), right

```

5.4 Choosing k via Cumulative Energy

```
energy = np.cumsum(S**2) / np.sum(S**2) # Eq. (20): energy(k) for every prefix k
k90 = np.searchsorted(energy, 0.90) + 1 # smallest k with energy(k) >= 0.90
k99 = np.searchsorted(energy, 0.99) + 1 # smallest k with energy(k) >= 0.99
```

`np.cumsum(S**2)` computes the running sum $\sum_{i=1}^k \sigma_i^2$ for every k at once; dividing by the total $\sum_i \sigma_i^2$ gives Equation (22) for every k simultaneously, and `searchsorted` finds the first index crossing the 90%/99% threshold.

```
def compress_color_image(rgb_array, k):
    channels = []
    for c in range(3):
        Uc, Sc, Vtc = svd_from_scratch(rgb_array[:, :, c])
        Uk, Sk, Vtk = compress(Uc, Sc, Vtc, k)
        channels.append(decompress(Uk, Sk, Vtk))
    out = np.clip(np.stack(channels, axis=-1), 0, 255)
    return out.astype(np.uint8)
```

Each channel gets its own singular value spectrum and its own optimal rank- k approximation, consistent with Equation (??).

6 Results

Running the pipeline on a 126×128 test image gives the singular value spectrum in Figure 1: it decays fast, so most of the image’s energy sits in the first handful of directions.

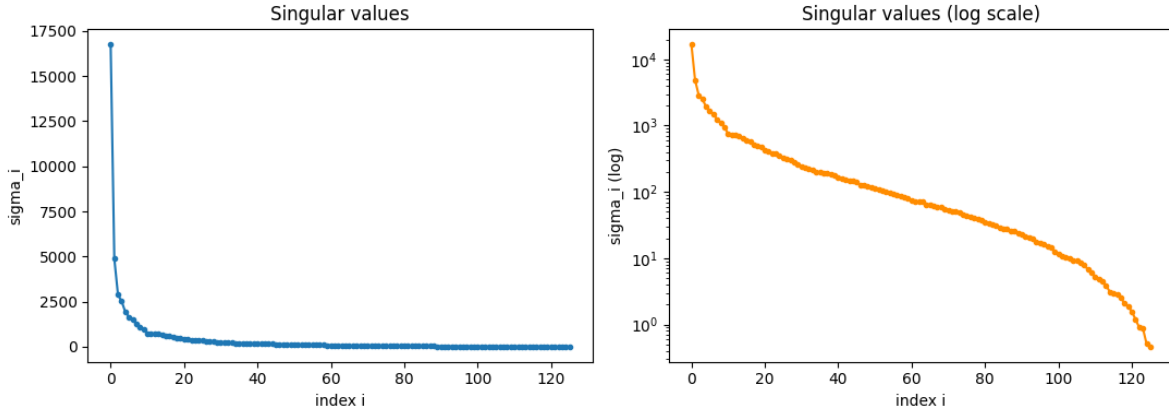


Figure 1: Singular values σ_i on a linear scale (left) and log scale (right). The steep initial drop is why low-rank truncation captures most of the image with a small k .

Using Equation (22), the rank needed to capture 90% and 99% of the energy is:

- 90% energy: $k = 2$ out of 126.
- 99% energy: $k = 16$ out of 126.

Figure 2 shows the reconstructed image at several values of k , alongside its compression ratio and PSNR.



Figure 2: Original image vs. rank- k reconstructions for $k = 2, 5, 10, 16, 20, 126$. As k grows the image sharpens and PSNR rises, at the cost of a smaller compression ratio.

k	Compression ratio	PSNR (dB)
2	$31.62\times$	15.04
5	$12.65\times$	18.57
10	$6.32\times$	22.47
16	$3.95\times$	25.34
20	$3.16\times$	27.20
126	$0.50\times$	296.53

Table 1: Compression ratio and PSNR as a function of k . At $k = 126$ (full rank for this 126×128 image) reconstruction is exact up to floating-point error, but storage is now *larger* than the original — reflecting Equation (20)’s crossover point.

7 Notebook Structure (Cell-by-Cell)

1. **Imports** — `numpy`, `matplotlib`, `PIL`; deliberately *no* `scipy.linalg` or `numpy.linalg` decomposition routines.
2. `jacobi_eigenvalue` — Algorithm 1, implementing Equations (11)–(17).
3. **Sanity check** on a random symmetric matrix $S = MM^T$: verifies $\|S - V\Lambda V^T\| \approx 0$ and $\|V^T V - I\| \approx 0$.
4. `svd_from_scratch` — Equations (7)–(10).
5. **Sanity check** on a random non-square matrix: verifies reconstruction error and orthogonality of U .
6. **Load image** — reads a user image or falls back to a synthetic test pattern; resizes to keep the $O(n^3)$ -per-sweep Jacobi method tractable.
7. **Run SVD** on the image matrix; reports timing and full-rank reconstruction error (should be ≈ 0 , confirming Equation (4)).
8. **Singular value spectrum** — linear and log-scale plots of σ_i ; computes k_{90}, k_{99} via Equation (22).
9. `compress` / `decompress` / `compression_ratio` / `psnr` — Equations (18), (19), (20), (21).
10. **Multi- k comparison** — reconstructs the image at several ranks and displays compression ratio + PSNR side by side.

11. **On-disk savings** — actually writes the truncated factors to `.npz` and compares file size against the raw array, confirming the theoretical ratio in Equation (20) holds in practice.

8 Why This Works: Intuition Recap

- An image matrix’s rows/columns are typically highly correlated (smooth regions, repeated textures, symmetric structure), so most of its ”shape” is captured by a small number of dominant singular directions.
- The Eckart–Young–Mirsky theorem guarantees truncated SVD is the *provably optimal* low-rank approximation under squared error — no other rank- k matrix does better in Frobenius norm.
- The Jacobi method computes this decomposition using only rotations — a sequence of numerically stable, orthogonality-preserving elementary operations — which is why it was chosen for a ”from scratch” implementation instead of, say, a naive power-iteration scheme.
- Compression ratio (Equation (20)) grows linearly better as k shrinks, while PSNR (Equation (21)) degrades — the classic rate–distortion trade-off, made concrete and tunable via a single integer k .

9 Practical Notes and Limitations

- **Complexity.** Each Jacobi sweep costs $O(n^3)$ (rotating all rows/columns for all $\binom{n}{2}$ pairs), and multiple sweeps are needed for convergence. This is far slower than LAPACK’s `dgesvd/dgesdd` (used internally by `np.linalg.svd`), which is why the notebook keeps images small (e.g. 128×128).
- **Numerical stability.** Because $A^T A$ squares the condition number of A (i.e. $\kappa(A^T A) = \kappa(A)^2$), computing SVD via $A^T A$ is less numerically robust than direct bidiagonalization-based SVD algorithms for ill-conditioned matrices. It is entirely adequate for 8-bit image data.
- **Rank-deficiency.** Singular values below 10^{-10} are treated as zero to avoid division-by-zero when recovering u_i ; this only discards directions that contribute negligibly to A .
- **Production use.** This implementation is for pedagogical transparency. For real workloads, always prefer `np.linalg.svd` (or a randomized SVD for very large matrices).

Reference

I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
Relevant source pages:

- Section 2.7, ”Eigendecomposition,” p. 42.
- Section 2.8, ”Singular Value Decomposition,” pp. 44–45.
- Section 2.12, ”Example: Principal Components Analysis,” pp. 48–51.